Step 3 — The Pipeline (final) — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — the one lambda that connects RAG to the matcher
- Hint 2 — run_match, in order

Hint 1 — the one lambda that connects RAG to the matcher

Step 1's `match_skills` wants a `distance(skill)` function. Step 2's `best_distance(collection, skill)` takes two arguments. Bridge them with a small closure over the collection:

```
def distance(skill):
    return best_distance(collection, skill)
```

Now `match_skills(required, distance, cutoff)` works — the matcher asks `distance(skill)` and gets a real resume-search result, without ever importing Chroma.

Hint 2 — run_match, in order

`run_match(model, resume_text, job_text, cutoff)` is just the three verified pieces, in order:

```
collection = build_resume_store(resume_text)          # Step 2
required   = extract_required_skills(model, job).skills # Step 2
report     = match_skills(required, distance, cutoff)   # Step 1
return report
```

`main.py` reads the resume file and the job file (two arguments), calls `run_match`, and prints the score, the matched skills, and the missing ones. The integration test runs a resume that covers Python/Docker/AWS/leadership against a job that also wants Kubernetes, and checks the score is high and "Kubernetes" is in `missing`. The whole file set is in the Solution.